# SIMATS ENGINEERING

# ASSESSMENT TOOL 3

**COURSE NAME:** **Database Management Systems**

**COURSE CODE:** **CSA0501**

---

**COURSE OUTCOME COVERED**

**CO4:** *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)

Assessment Tool Weightage: 10%

---

| **QUESTIONS** | | |
|---|---|---|
| **Q.No** | **Question** | **Bloom's Level** |
| **1** | Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency. | Bloom's Level: 4 – Analyze |
| **2** | Differentiate between clustered indexing and non-clustered indexing with suitable database examples. | Bloom's Level: 4 – Analyze |
| **3** | Compare primary indexing and secondary indexing based on storage requirements and search performance. | Bloom's Level: 4 – Analyze |
| **4** | Analyze the differences between static hashing and dynamic hashing in database systems. | Bloom's Level: 4 – Analyze |
| **5** | Compare B-Tree indexing and B+ Tree indexing for large-scale database applications. | Bloom's Level: 4 – Analyze |

| | | |
|---|---|---|
| **6** | Differentiate linear hashing and extendible hashing in terms of bucket management and scalability. | Bloom's Level: 4 – Analyze |
| **7** | Compare dense indexing and sparse indexing with respect to memory usage and access speed. | Bloom's Level: 4 – Analyze |
| **8** | Analyze the advantages and limitations of indexed sequential file organization versus direct file organization. | Bloom's Level: 4 – Analyze |
| **9** | Compare single-level indexing and multi-level indexing for efficient database retrieval operations. | Bloom's Level: 4 – Analyze |
| **10** | Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases. | Bloom's Level: 5 – Evaluate |

---

***Code of Conduct:***

*I D. Gnana Deepthi(192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

# ANSWERS

## 1. Sequential File Organization vs Heap File Organization

| Aspect | Sequential File Organization | Heap File Organization |
|---|---|---|
| **Insertion** | Slower because records must be inserted in the correct sequence/order. | Faster because new records can be added wherever free space is available, usually at the end. |
| **Deletion** | Slower because deleting a record may require rearranging records or maintaining the sequence. | Faster because the record can simply be marked as deleted or removed. |
| **Retrieval** | Very efficient for **sequential access** and range-based searches because records are ordered. | Efficient for retrieving all records, but slow for searching for a particular record without an index. |
| **Search** | Faster when searching in the ordering field; binary search can be used. | Usually requires a linear search, making individual searches slower. |
| **Maintenance** | Requires maintaining the sorted order, which increases overhead. | Simple to maintain because no ordering is required. |
| **Best suited for** | Applications requiring sequential processing and range queries. | Applications with frequent insertions and simple record storage. |

## 2. Clustered Indexing vs Non-Clustered Indexing

| Feature | Clustered Indexing | Non-Clustered Indexing |
|---|---|---|
| **Data Storage** | Determines the physical order of records in the table. | Does not change the physical order of records. |
| **Number per table** | Usually only **one** clustered index because data can have only one physical order. | Multiple non-clustered indexes can be created. |
| **Search Speed** | Very fast for range queries and sequential access. | Fast for specific searches but may require an additional lookup to fetch the actual row. |

| Feature | Clustered Indexing | Non-Clustered Indexing |
|---|---|---|
| **Storage** | Index is closely associated with the actual data storage. | Index is stored separately and contains pointers/references to the actual records. |
| **Range Queries** | Highly efficient. | Generally less efficient than clustered indexing for large ranges. |
| **Example** | Student(Student_ID, Name, Department, CGPA) with a clustered index on Student_ID. | Same table with a non-clustered index on Department. |

**Example**

Suppose we have:

| Student_ID | Name | Department | CGPA |
|---|---|---|---|
| 101 | Ravi | CSE | 8.5 |
| 102 | Anu | ECE | 9.1 |
| 103 | Kiran | CSE | 8.8 |
| 104 | Priya | IT | 9.3 |

**Clustered index on Student_ID:**

- The actual student records are stored in Student_ID order.
- Searching for students with IDs from 101 to 103 is efficient.

**Non-clustered index on Department:**

- A separate index stores department values and references to the corresponding records.
- Searching for all students in CSE can use this index without physically rearranging the table.

**Conclusion:** Clustered indexing is suitable when data is frequently accessed in sorted or range order, while non-clustered indexing is useful for supporting multiple search conditions on different columns.

## 3. Primary Indexing vs Secondary Indexing

| Feature | Primary Indexing | Secondary Indexing |
|---|---|---|
| **Definition** | Index created on the primary key or ordering key of a sorted file. | Index created on a non-ordering field of a file. |
| **Data Ordering** | Data records are physically ordered according to the index field. | Data records are not necessarily ordered according to the indexed field. |
| **Storage Requirement** | Requires less storage because it is usually a **sparse index** with one index entry per block. | Requires more storage because it is generally a **dense index** with entries for records or groups of records. |
| **Search Performance** | Very fast because the index is small and binary search can be applied efficiently. | Fast, but may require additional pointer/reference lookups to locate the actual record. |
| **Number of Indexes** | Usually one primary index for a sorted file. | Multiple secondary indexes can be created on different attributes. |
| **Example** | Index on Student_ID when records are sorted by Student_ID. | Index on Department or CGPA when records are not physically ordered by these fields. |

**Example**

Consider a **Student** table:

| Student_ID | Name | Department | CGPA |
|---|---|---|---|
| 101 | Ravi | CSE | 8.5 |
| 102 | Anu | ECE | 9.1 |
| 103 | Kiran | CSE | 8.8 |
| 104 | Priya | IT | 9.3 |

**Primary Index:**
If records are sorted by Student_ID, the primary index may contain:

101 → Block 1

103 → Block 2

Since one index entry can represent a block, it requires **less storage**.

**Secondary Index:**
If an index is created on Department, it may contain:

CSE → 101, 103

ECE → 102

IT → 104

This requires **more storage**, especially when many records have the same value.

## 4. Static Hashing vs Dynamic Hashing

| Feature | Static Hashing | Dynamic Hashing |
|---|---|---|
| **Definition** | Uses a fixed number of buckets throughout the file's lifetime. | Number of buckets can increase or decrease as the data changes. |
| **Bucket Size** | Fixed. | Can grow dynamically. |
| **Handling Growth** | Performance may decrease when the file grows. | Handles database growth efficiently. |
| **Collisions** | More collisions may occur as buckets become full. | Reduces collisions by splitting or adding buckets. |
| **Overflow** | Uses overflow buckets when the main bucket becomes full. | Usually minimizes overflow through bucket splitting. |
| **Performance** | Good for databases with a relatively fixed size. | Better for databases with frequently changing sizes. |
| **Storage** | Simple and requires less management overhead. | Requires additional storage and management for dynamic structures. |
| **Complexity** | Easy to implement and maintain. | More complex to implement. |
| **Example** | A student database with a fixed number of 100 buckets. | An e-commerce database where customer records continuously increase. |

**Example**

Suppose a hash function is:

h(key) = key % 5

In **static hashing**, there are always **5 buckets**. If many records map to the same bucket, overflow buckets are created.

In **dynamic hashing**, new buckets can be created when existing buckets become full. The records are redistributed to maintain efficient access.

**Advantages and Disadvantages**

**Static Hashing**

- **Advantages:** Simple, fast, and low management overhead.
- **Disadvantages:** Performance decreases as the database grows due to overflow and collisions.

**Dynamic Hashing**

- **Advantages:** Adapts to database growth and reduces overflow problems.
- **Disadvantages:** More complex and requires additional overhead for bucket splitting and restructuring.

## 5. B-Tree vs B+ Tree Indexing

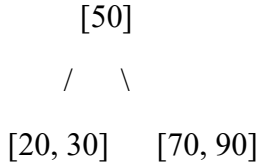
| Feature | B-Tree | B+ Tree |
|---|---|---|
| **Data Storage** | Stores keys and actual record pointers/data in both internal and leaf nodes. | Stores keys in internal nodes and record pointers/data only in leaf nodes. |
| **Search** | Search may finish at an internal node. | Search always proceeds to a leaf node. |
| **Range Queries** | Less efficient. | **Highly efficient** because leaf nodes are linked. |
| **Sequential Access** | Comparatively slower. | Faster due to linked leaf nodes. |
| **Fan-out** | Lower because internal nodes store data pointers. | Higher because internal nodes store only keys and pointers. |
| **Tree Height** | May be slightly greater. | Usually smaller due to higher fan-out. |
| **Disk I/O** | More disk accesses may be required. | Fewer disk accesses are generally required. |

| Feature | B-Tree | B+ Tree |
|---|---|---|
| **Complexity** | Relatively more complex for range traversal. | Efficient and well suited for range traversal. |
| **Large Databases** | Suitable, but less commonly preferred. | **Widely preferred for large-scale database indexing.** |

**Example**

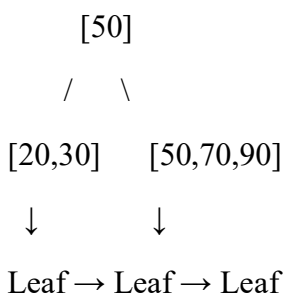
Consider an employee database indexed by Employee_ID.

**B-Tree:**




Keys and record pointers can exist in both internal and leaf nodes.

**B+ Tree:**




All actual record pointers are stored at the **leaf level**, and the leaf nodes are linked. Therefore, a query such as:

SELECT * FROM Employee

WHERE Employee_ID BETWEEN 50 AND 90;

can be performed efficiently by locating the first matching leaf and then following the linked leaf nodes.

## 6. Linear Hashing vs Extendible Hashing

| Feature | Linear Hashing | Extendible Hashing |
|---|---|---|
| **Bucket Management** | Buckets are split **gradually and sequentially**. | Buckets are split based on hash values, and a **directory** is used to manage them. |
| **Directory** | Does not require a directory. | Requires a directory containing pointers to buckets. |
| **Bucket Splitting** | One bucket is split at a time according to a predefined split order. | A bucket is split when it overflows; the directory may also need to expand. |
| **Scalability** | Grows smoothly without sudden restructuring. | Can grow rapidly by doubling the directory size when required. |
| **Memory Overhead** | Lower because no directory is maintained. | Higher because of the directory. |
| **Overflow Handling** | Overflow chains may temporarily occur before a bucket is split. | Bucket splitting generally reduces overflow quickly. |
| **Complexity** | Relatively simple to implement. | More complex because of directory management. |
| **Best Suited For** | Databases that grow continuously and need gradual expansion. | Applications requiring very fast direct access and efficient overflow handling. |

## 7. Dense Indexing vs Sparse Indexing

| Feature | Dense Indexing | Sparse Indexing |
|---|---|---|
| **Index Entries** | Contains an index entry for **every record**. | Contains index entries for **only some records**, usually one per block. |
| **Memory Usage** | **Higher**, because many index entries are stored. | **Lower**, because fewer entries are stored. |
| **Access Speed** | Faster because the index directly identifies the required record. | Slightly slower because after finding the appropriate index entry, the block may need to be searched. |

| Feature | Dense Indexing | Sparse Indexing |
|---|---|---|
| **Search** | Very efficient for individual record searches. | Efficient, but may require additional sequential searching within a block. |
| **Storage Requirement** | Requires more disk space. | Requires less disk space. |
| **Maintenance** | More costly to update when records are inserted or deleted. | Easier to maintain because fewer index entries exist. |
| **Example** | An index entry for every Student_ID. | One index entry for each data block containing the first Student_ID. |

## 8. Indexed Sequential File Organization vs Direct File Organization

| Feature | Indexed Sequential File Organization | Direct File Organization |
|---|---|---|
| **Access Method** | Supports both sequential and direct access using an index. | Provides direct access using a hashing function or address calculation. |
| **Search Speed** | Fast because an index helps locate the required record. | Very fast for exact-match searches because the record address is calculated directly. |
| **Sequential Access** | **Highly efficient** because records are stored in sorted order. | Less efficient for sequential processing because records may not be stored sequentially. |
| **Range Queries** | **Excellent** for range queries. | Generally poor for range queries because hashing does not preserve order. |
| **Insertion** | Can be slower because the sorted order must be maintained. | Usually fast, although collisions may require additional handling. |
| **Deletion** | More complex because indexes and file ordering may need maintenance. | Relatively simple, but deleted spaces may need to be managed. |
| **Storage Overhead** | Requires additional storage for indexes and overflow areas. | Requires less index overhead but may require overflow space for collisions. |

| Feature | Indexed Sequential File Organization | Direct File Organization |
|---|---|---|
| **Scalability** | Suitable for large files, but index maintenance increases as data grows. | Scales well for exact-match operations if the hash function distributes records evenly. |
| **Main Limitation** | Index and overflow-area maintenance can be expensive. | Cannot efficiently support ordered or range-based searches. |

## 9. Single-Level Indexing vs Multi-Level Indexing

| Feature | Single-Level Indexing | Multi-Level Indexing |
|---|---|---|
| **Definition** | Uses one index table to locate records in the data file. | Uses multiple levels of indexes, where one index points to another index. |
| **Search Speed** | Fast for small indexes, but slows down when the index becomes large. | Faster for large databases because higher-level indexes reduce the search space. |
| **Memory Usage** | Requires less memory for small files. | Requires additional storage for multiple index levels. |
| **Disk I/O** | May require more disk accesses for a large index. | Requires fewer disk accesses because the search is performed hierarchically. |
| **Index Size** | Can become very large as the number of records increases. | Each higher-level index is smaller than the level below it. |
| **Scalability** | Suitable for small and medium-sized databases. | Highly suitable for large-scale databases. |
| **Complexity** | Simple to implement and maintain. | More complex to implement and maintain. |
| **Example** | One index directly points to data blocks. | Level 1 points to Level 2, which points to the data blocks. |

## 10. Hashing vs Indexing for Exact-Match and Range Queries

Hashing and indexing are important techniques used to improve database retrieval performance. Their efficiency depends on whether the query is an exact-match query or a range query.

| Feature | Hashing | Indexing |
|---|---|---|
| **Exact-Match Query** | **Very fast**, usually close to O(1) average time. | Fast, generally O(log n) with B/B+ Tree indexes. |
| **Range Query** | **Poor**, because hash values do not preserve the order of keys. | **Very efficient**, especially with B+ Tree indexes. |
| **Data Ordering** | Does not maintain ordering. | B/B+ Tree indexes maintain sorted key order. |
| **Collision Handling** | Required when multiple keys map to the same bucket. | No hash collisions; tree structure manages keys. |
| **Insertion** | Fast on average. Dynamic hashing handles growth efficiently. | Efficient, but tree restructuring may be required. |
| **Storage** | Requires buckets and possibly overflow space. | Requires additional space for index nodes and pointers. |
| **Best Use** | Exact searches such as WHERE Student_ID = 101. | Exact and range searches such as WHERE CGPA BETWEEN 8.0 AND 9.0. |
| **Large Database Performance** | Excellent for exact-match operations. | Excellent for both exact-match and range operations. |

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage (%) | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Scope of Items | 20% | Selects highly relevant items with clear scope and justification | Selects appropriate items with minor gaps | Selection is limited or partially relevant | Items are unclear or not relevant |
| Comparison Depth | 25% | Provides detailed and comprehensive comparison across multiple aspects | Provides adequate comparison with some depth | Limited comparison with few aspects | Very minimal or no comparison |
| Use of Evidence | 20% | Supports comparison with accurate data, examples, or references | Uses some supporting evidence with minor gaps | Limited or weak supporting evidence | No supporting evidence provided |
| Critical Thinking | 20% | Demonstrates strong critical thinking with clear interpretation and reasoning | Shows reasonable interpretation with minor gaps | Basic interpretation; lacks depth | No meaningful interpretation |
| Clarity of Presentation | 15% | Well-organized, clear, and logically structured presentation | Generally clear with minor issues | Somewhat unclear or poorly structured | Disorganized and difficult to understand |